



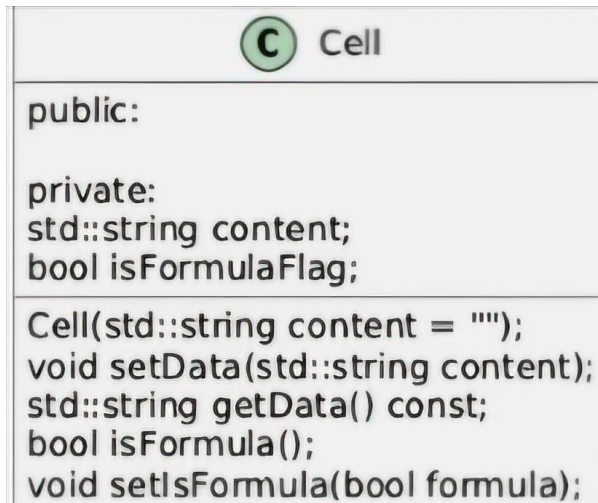
# CSE 241 – OBJECT ORIENTED PROGRAMMING HOMEWORK #1

Zeynep Etik

Student ID 220104004035

In this project I created three different classes called SpreadSheet, Cell and FormulaParser whose UML diagrams are shown in the below.

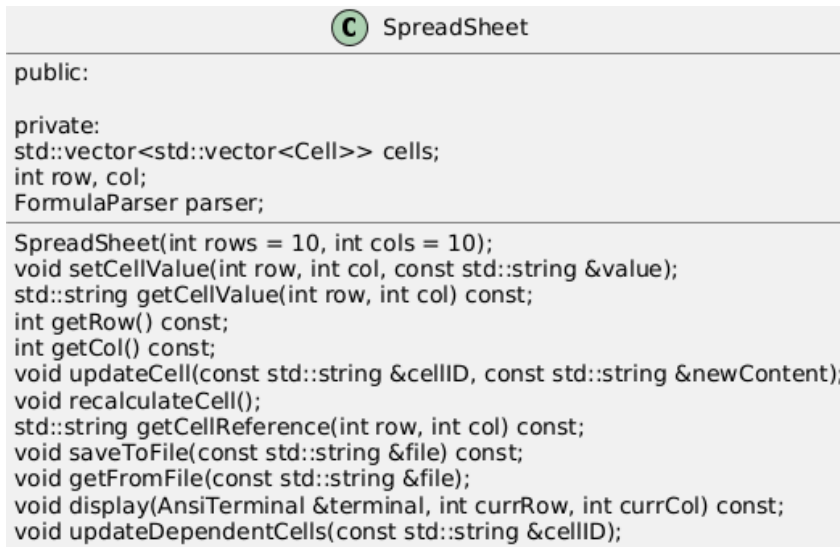
### Diagram and Description of Cell class



This class has two private data. Content member data is for storing cell formulas or values. And isFormul is for storing the information that cell has formula in it or a value.

For the public there are two set functions which are setData() and setIsFormula() for setting informations in cell. And two get functions which are getData() and isFormula() to get the content and to decide whether it's formula or not respectively.

## Diagram of SpreadSheet class



SpreadSheet class has three private data members. **cells** vector whose type is **Cell**. To be able to do operations on spreadsheet we will use this vector. And to store dimensions of the vector there is two data variables called **row** and **col**. There is another data variable whose type is **FormulaParser** called **parser**. **parser** data variable helps us to call **FormulaParser** functions within the **SpreadSheet** class.

For the public part firstly, there is a constructor to create the **cells** vector and initiate **row** and **col** values.

`void SpreadSheet::setCellValue(int r, int c, const string& data)`

There is a set function named **setCellValue()** for storing a string in the desired location of the table.

### Get Functions

There are three get functions; one is to get row value, another one is for getting column value, and lastly for getting the content of the cell a string.

**\*\*The functions with highlighted with asterisk are not used. Since I implement a function for displaying after writing them.**

**Because I worked hard on them I wanted to add them also.**

**\*void SpreadSheet::updateCell(const string cellID, const string& newContent)**

To update the content of a specific cell, updateCell() function is called. This function gets two parameters one for cell ID such as A5, and another for new content of the cell. This function gets the column and row of the cell whose Id is given by calling getRowIndex() and getColIndex() these functions gets the cell ID as their parameters and returns row and column values as integers. Go back to updateCell() function, after getting row and column values it will change the cell stores if the new content is formula or not by using isFormula() function of Cell class. If it is formula it converts the formula into its numerical value and lastly stores it by using Cell class's function setData(). At the end of the function it will call recalculate() function to change dependent cells within the table.

**\*void SpreadSheet::recalculateCell()**

recalculate() function controls each cells and if there's a formula inside the cell, it updates cells value.

**\*void SpreadSheet::updateDependentCells(const string& cellID)**

This function gets the cell ID as a parameter and controls over the cells if there's a cell that its data contains that cell ID. If contains then it will get the formula inside the cell and call

parseFormula() function from FormulaParser by using parser data variable.

`string Spreadsheet::getCellReference(int r, int c) const`

There is also another get function to get cell ID from row and column. It converts column which is integer to its letter form and add this letter with row which will be converted into a string by using to\_string() function from string library.

`void Spreadsheet::saveToFile(const string& fname) const`

It gets the filename as a string and create an object called file. Then controls whether it is open or not if not it will give an error message.

If file is open then starts to scan cell values if cell value is empty it will add a space character. If it is not empty it will add the cell's content into the file. And after every cell it will add an comma after cell's content. If inner dimension is done it will add an \n character and do the same things for the next row.

At the end it will close the file.

`void Spreadsheet::getFromFile(const string& fname)`

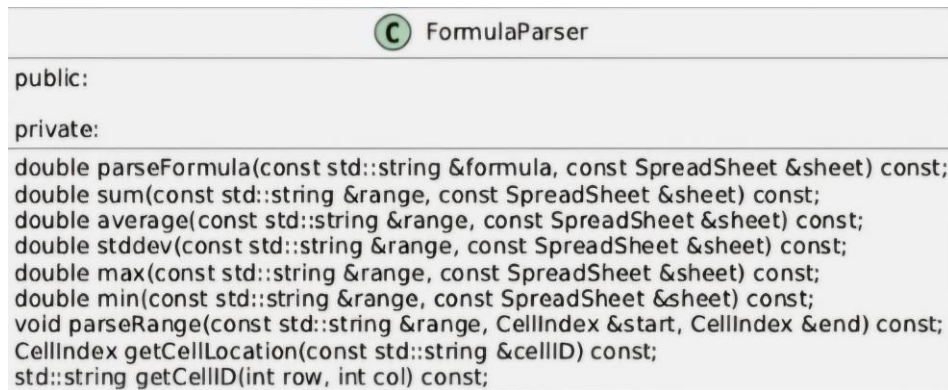
This function also gets a string as a parameter which is named fname. Then creates an object named file. Then controls if it is open. If it is open, clears all the cells. Then it starts to take lines and separate this line into characters and controls all the characters if character is a comma. It will add the cell's content into a vector and will clear the string that stores the cell's

content for next ones. If character is not a comma then it will add the character into a string variable. At the end of the line it will add the last cell's content. And add that 1D vector into a temporary 2D vector. At the end of the file it will store row and column numbers into SpreadSheet's private data variables. Then resize the cells vector which is also a private data variable of SpreadSheet. And cell's contents will be added into cells vector by using loops. At the end file will be called.

```
void SpreadSheet::display(AnsiTerminal &terminal, int currentRow, int currentCol) const
```

Firstly this function clears the screen by using provided functions in AnsiTerminal. Then will print the instructions. Then gets the cell reference and cell value to print the selected cell and its content. Then it will print the column headers by using provided functions in AnsiTerminal. Then row headers will be printed and it will get into another loop to print contents or values of the cells if current row and column that user selected is same as with loops row and column it will printed it in a different colour to show its speciality. And if there is no content it will print space if not it will print cell's value. if loop's row and column are different then it will print period.

## Diagram of FormulaParser class



\*\*I did not understand but plantUML shows all functions into the private. But parseRange(), getCellLocation() and getCellID() are in the private.

`CellIndex getCellLocation(const string& cellID)const;`

This function converts the given cell ID into row and columns. And returns a struct called CellIndex.

`string getCellID(int r, int c)const;`

This function converts given row and columns into cell ID. And returns it as string.

`void parseRange(const string& range, CellIndex& start, CellIndex& end)const;`

This function will give the row and column values of range's end and start cells. It tries to find the .. characters because these characters separates the end and start cells of the range. If it cannot find .. characters it will return zero for both row and column.

```
double parseFormula(const string& Formula, const SpreadSheet& sheet)const;
```

First of all it controls if given string is really a formula if not it will return zero.

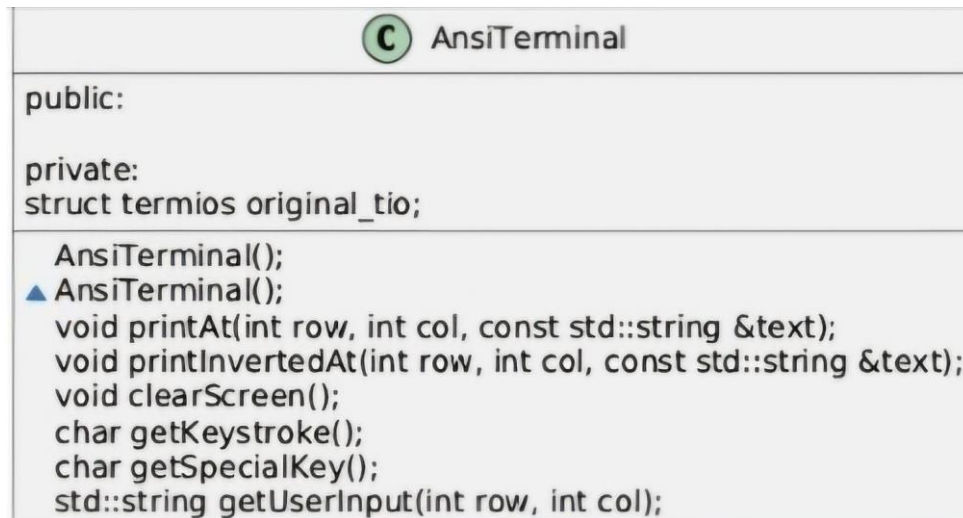
Then to store cell IDs it will declare two strings and a character for the operator. It will control if it contains keywords such as SUM, AVER, etc. if these operations are called it will find the range and calls the related function and returns their values.

If not it will control if there is letter or number and add these characters into the string that stores cell's content in it. Until it encounters an operator. And do the same things for operator and second cell or value. Then to be able to do the mathematical operations it will convert them into doubles and according to the operator it will calculate and return the value.

### SUM, AVER, STDDEV, MIN, MAX, Operations

For all of these operations functions. First it declares two variables whose type is CellIndex named start and end. These variables symbolize start cell and end cell of the range. And to parse the range it will call parseRange() function. And there will be a nested loop to get the cell's values and do the calculations of each operation. And they will return calculation's result.

## Diagram for the the given class AnsiTerminal



I just added `getUserInput()` function into the `AnsiTerminal` class to be able to make user to see what they are writing. And I got it from gpt. But since my ubuntu terminal had some problems with enter key or `'\n'` I could not use it to end the loop. So instead using enter key it uses `^` character to enter the desired value.

## Main Function

In the `main()` function firstly it asks for desired row and col values and create a `SpreadSheet` object named as `sheet` according to the desired row and colmuns. Then starts a while loop by using boolean variable. It uses up, down, left and right arrows to move in the table. According to users command if user wants to edit the table it will get users input and call `setCellValue()` function. For file operations it will get the file name and call the save or load functions. And after every operation it will call `display()` function from `SpreadSheet` class. And if users want to quit it will change the boolean's value. At

the end of the program screen will be cleared and there will be goodbye message.

## Missing Parts of the program

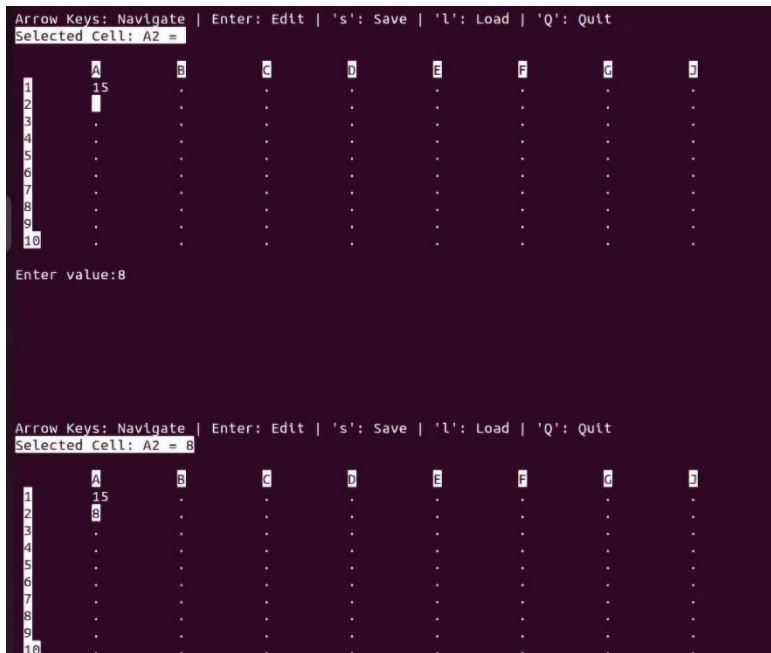
For formulas if there are cell IDs more than two such as  $=A1+B5*C2$  it will end the program. I did not use `recalculate()` or `updateDependentCells()` functions to calculate the dependent cell's value. Since I declared a display function it calculates every cell's value in every iteration. And the values are not printed as doubles I discovered it too late

## In Which Parts Did I Use AI Assistance

For loading files I got the help of Chatgpt because tried to put data into the cell directly without any helper vector etc. In the `display()` function I had some problems also so I got the help of AI. In the `parseRange()` function I had some problems with delimiter so in there I got the AI assistance. And lastly at the main function because at first I could not figure out how to move in the terminal.

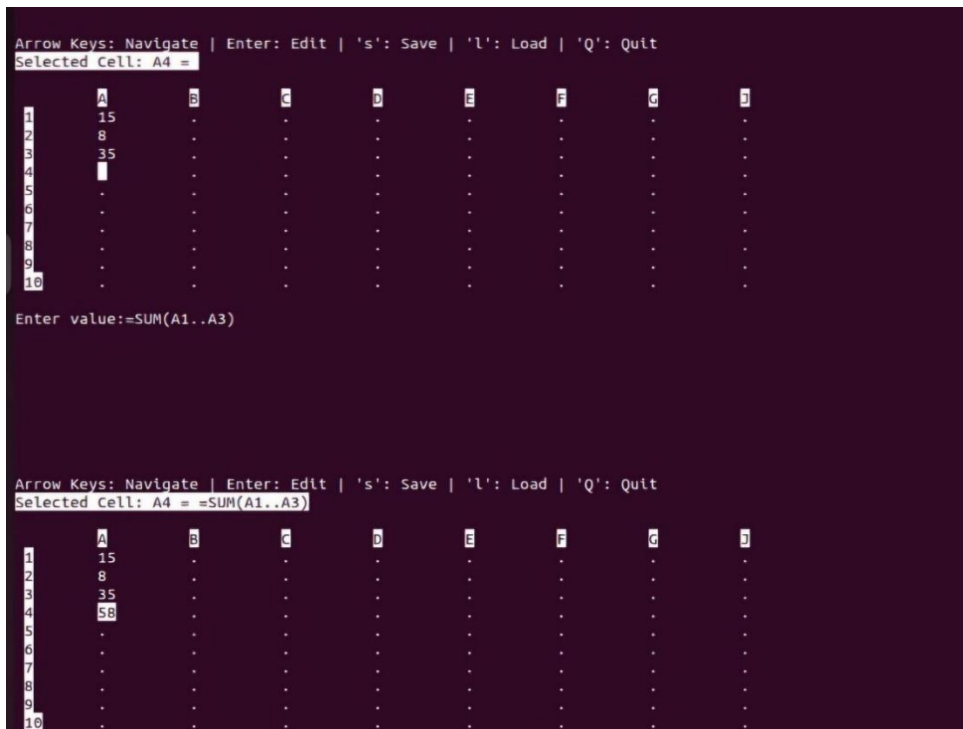
## User Guide

User can move by using right, left, up and down arrows of the keyboard. If they want to edit a specific cell they should enter 'E' character.



If they want to add an formula (such as A1+14, SUM(B1..B4), etc.) they should insert '=' character then write the formula. Formulas should be written in capital letters. Formats of the formulas should be as shown below:

SUM(A1..A7)



## AVER(A1..A7)

Arrow Keys: Navigate | Enter: Edit | 's': Save | 'l': Load | 'Q': Quit  
Selected Cell: C4 =

|    | A  | B  | C | D  | E | F | G | H |
|----|----|----|---|----|---|---|---|---|
| 1  | 15 | .  | . | .  | . | . | . | . |
| 2  | 8  | .  | . | .  | . | . | . | . |
| 3  | 35 | .  | . | .  | . | . | . | . |
| 4  | 58 | .  | . | .  | . | . | . | . |
| 5  | .  | .  | . | 32 | . | . | . | . |
| 6  | .  | .  | . | .  | . | . | . | . |
| 7  | .  | .  | . | .  | . | . | . | . |
| 8  | .  | 24 | . | .  | . | . | . | . |
| 9  | .  | .  | . | .  | . | . | . | . |
| 10 | .  | .  | . | .  | . | . | . | . |

Enter value: =AVER(A2..A4)

Arrow Keys: Navigate | Enter: Edit | 's': Save | 'l': Load | 'Q': Quit  
Selected Cell: C4 = =AVER(A2..A4)

|    | A  | B  | C  | D  | E | F | G | H |
|----|----|----|----|----|---|---|---|---|
| 1  | 15 | .  | .  | .  | . | . | . | . |
| 2  | 8  | .  | .  | .  | . | . | . | . |
| 3  | 35 | .  | .  | .  | . | . | . | . |
| 4  | 58 | .  | 33 | .  | . | . | . | . |
| 5  | .  | .  | .  | 32 | . | . | . | . |
| 6  | .  | .  | .  | .  | . | . | . | . |
| 7  | .  | .  | .  | .  | . | . | . | . |
| 8  | .  | 24 | .  | .  | . | . | . | . |
| 9  | .  | .  | .  | .  | . | . | . | . |
| 10 | .  | .  | .  | .  | . | . | . | . |

## MIN(A1..A7)

Arrow Keys: Navigate | Enter: Edit | 's': Save | 'l': Load | 'Q': Quit  
Selected Cell: B2 =

|    | A  | B  | C  | D  | E | F | G | H |
|----|----|----|----|----|---|---|---|---|
| 1  | 15 | .  | .  | .  | . | . | . | . |
| 2  | 8  | .  | .  | .  | . | . | . | . |
| 3  | 35 | .  | .  | .  | . | . | . | . |
| 4  | 58 | .  | 33 | .  | . | . | . | . |
| 5  | .  | .  | .  | 32 | . | . | . | . |
| 6  | .  | .  | .  | .  | . | . | . | . |
| 7  | .  | .  | .  | .  | . | . | . | . |
| 8  | .  | 24 | .  | .  | . | . | . | . |
| 9  | .  | .  | .  | .  | . | . | . | . |
| 10 | .  | .  | .  | .  | . | . | . | . |

Enter value: =MIN(A1..A4)

Arrow Keys: Navigate | Enter: Edit | 's': Save | 'l': Load | 'Q': Quit  
Selected Cell: B2 = =MIN(A1..A4)

|    | A  | B  | C  | D  | E | F | G | H |
|----|----|----|----|----|---|---|---|---|
| 1  | 15 | .  | .  | .  | . | . | . | . |
| 2  | 8  | 8  | .  | .  | . | . | . | . |
| 3  | 35 | .  | .  | .  | . | . | . | . |
| 4  | 58 | .  | 33 | .  | . | . | . | . |
| 5  | .  | .  | .  | 32 | . | . | . | . |
| 6  | .  | .  | .  | .  | . | . | . | . |
| 7  | .  | .  | .  | .  | . | . | . | . |
| 8  | .  | 24 | .  | .  | . | . | . | . |
| 9  | .  | .  | .  | .  | . | . | . | . |
| 10 | .  | .  | .  | .  | . | . | . | . |

## MAX(A1..A7)

Arrow Keys: Navigate | Enter: Edit | 's': Save | 'l': Load | 'Q': Quit

Selected Cell: A6 =

|    | A  | B  | C  | D  | E | F | G | H |
|----|----|----|----|----|---|---|---|---|
| 1  | 15 | .  | .  | .  | . | . | . | . |
| 2  | 8  | 8  | .  | .  | . | . | . | . |
| 3  | 35 | .  | .  | .  | . | . | . | . |
| 4  | 58 | .  | 33 | .  | . | . | . | . |
| 5  | .  | .  | .  | 32 | . | . | . | . |
| 6  | .  | .  | .  | .  | . | . | . | . |
| 7  | .  | .  | .  | .  | . | . | . | . |
| 8  | .  | 24 | .  | .  | . | . | . | . |
| 9  | .  | .  | .  | .  | . | . | . | . |
| 10 | .  | .  | .  | .  | . | . | . | . |

Enter value: =MAX(A1..A3)

Arrow Keys: Navigate | Enter: Edit | 's': Save | 'l': Load | 'Q': Quit

Selected Cell: A6 = =MAX(A1..A3)

|    | A  | B  | C  | D  | E | F | G | H |
|----|----|----|----|----|---|---|---|---|
| 1  | 15 | .  | .  | .  | . | . | . | . |
| 2  | 8  | 8  | .  | .  | . | . | . | . |
| 3  | 35 | .  | .  | .  | . | . | . | . |
| 4  | 58 | .  | 33 | .  | . | . | . | . |
| 5  | .  | .  | .  | 32 | . | . | . | . |
| 6  | 35 | .  | .  | .  | . | . | . | . |
| 7  | .  | .  | .  | .  | . | . | . | . |
| 8  | .  | 24 | .  | .  | . | . | . | . |
| 9  | .  | .  | .  | .  | . | . | . | . |
| 10 | .  | .  | .  | .  | . | . | . | . |

## STDDEV(A1..A7)

Arrow Keys: Navigate | Enter: Edit | 's': Save | 'l': Load | 'Q': Quit

Selected Cell: B5 =

|    | A  | B  | C  | D  | E | F | G | H |
|----|----|----|----|----|---|---|---|---|
| 1  | 15 | .  | .  | .  | . | . | . | . |
| 2  | 8  | 8  | .  | .  | . | . | . | . |
| 3  | 35 | .  | .  | .  | . | . | . | . |
| 4  | 58 | .  | 33 | .  | . | . | . | . |
| 5  | .  | .  | .  | 32 | . | . | . | . |
| 6  | 35 | .  | .  | .  | . | . | . | . |
| 7  | .  | .  | .  | .  | . | . | . | . |
| 8  | .  | 24 | .  | .  | . | . | . | . |
| 9  | .  | .  | .  | .  | . | . | . | . |
| 10 | .  | .  | .  | .  | . | . | . | . |

Enter value: =STDDEV(A1..A4)

Arrow Keys: Navigate | Enter: Edit | 's': Save | 'l': Load | 'Q': Quit

Selected Cell: B5 = =STDDEV(A1..A4)

|    | A  | B  | C  | D  | E | F | G | H |
|----|----|----|----|----|---|---|---|---|
| 1  | 15 | .  | .  | .  | . | . | . | . |
| 2  | 8  | 8  | .  | .  | . | . | . | . |
| 3  | 35 | .  | .  | .  | . | . | . | . |
| 4  | 58 | .  | 33 | .  | . | . | . | . |
| 5  | .  | 19 | .  | 32 | . | . | . | . |
| 6  | 35 | .  | .  | .  | . | . | . | . |
| 7  | .  | .  | .  | .  | . | . | . | . |
| 8  | .  | 24 | .  | .  | . | . | . | . |
| 9  | .  | .  | .  | .  | . | . | . | . |
| 10 | .  | .  | .  | .  | . | . | . | . |

## A1+D5

Arrow Keys: Navigate | Enter: Edit | 's': Save | 'l': Load | 'Q': Quit  
Selected Cell: D5 =

|    | A  | B  | C | D | E | F | G | H |
|----|----|----|---|---|---|---|---|---|
| 1  | 15 | -  | - | - | - | - | - | - |
| 2  | 8  | -  | - | - | - | - | - | - |
| 3  | 35 | -  | - | - | - | - | - | - |
| 4  | 58 | -  | - | - | - | - | - | - |
| 5  | -  | -  | - |   | - | - | - | - |
| 6  | -  | -  | - | - | - | - | - | - |
| 7  | -  | -  | - | - | - | - | - | - |
| 8  | -  | 24 | - | - | - | - | - | - |
| 9  | -  | -  | - | - | - | - | - | - |
| 10 | -  | -  | - | - | - | - | - | - |

Enter value:=A2+B8

Arrow Keys: Navigate | Enter: Edit | 's': Save | 'l': Load | 'Q': Quit  
Selected Cell: D5 = =A2+B8

|    | A  | B  | C | D  | E | F | G | H |
|----|----|----|---|----|---|---|---|---|
| 1  | 15 | -  | - | -  | - | - | - | - |
| 2  | 8  | -  | - | -  | - | - | - | - |
| 3  | 35 | -  | - | -  | - | - | - | - |
| 4  | 58 | -  | - | -  | - | - | - | - |
| 5  | -  | -  | - | 32 | - | - | - | - |
| 6  | -  | -  | - | -  | - | - | - | - |
| 7  | -  | -  | - | -  | - | - | - | - |
| 8  | -  | 24 | - | -  | - | - | - | - |
| 9  | -  | -  | - | -  | - | - | - | - |
| 10 | -  | -  | - | -  | - | - | - | - |

## A1+15 etc.

And to store the data written they should insert “^” character (I had some problems with ENTER key on ubuntu and I could not figure it out why even after asking Chatgpt).

If user wants to save the file they should insert “s” command and then “^” character. After inserting command, it will ask for the file name write it out and insert “^” to run the program.

Arrow Keys: Navigate | Enter: Edit | 's': Save | 'l': Load | 'Q': Quit  
Selected Cell: C5 =

|    | A  | B  | C   | D  | E | F | G | H |
|----|----|----|-----|----|---|---|---|---|
| 1  | 15 | -  | -   | -  | - | - | - | - |
| 2  | 8  | 8  | 264 | -  | - | - | - | - |
| 3  | 35 | -  | -   | -  | - | - | - | - |
| 4  | 58 | -  | 33  | -  | - | - | - | - |
| 5  | -  | 19 |     | 32 | - | - | - | - |
| 6  | 35 | -  | -   | -  | - | - | - | - |
| 7  | -  | -  | -   | -  | - | - | - | - |
| 8  | -  | 24 | -   | -  | - | - | - | - |
| 9  | -  | -  | -   | -  | - | - | - | - |
| 10 | -  | -  | -   | -  | - | - | - | - |

Enter file name to save:  
Enter value:saved

## Display of saved file

```
15, , , , , , , ,
4, , , , , , , ,
, -SUM(A1..A2), , , , , ,
, -A1*C3, , , , , ,
, , , , , , , ,
, , , , , , , ,
, , , , , , , ,
, , , , , , , ,
, , , , , , , ,
, , , , , , , ,
```

If they want to load a file they should insert “l” command and then “^” character. After writing command, it will ask for the file name write the file name and insert “^” command.

```
15 8 264 33 32
35 19 32
35 24
Enter file name to load:
Loaded successfully!

Arrow Keys: Navigate | Enter: Edit | 's': Save | 'l': Load | 'Q': Quit
Selected Cell: C6 =
12 16
21
3 19
36
```

To end the program insert “Q” command and then “^” character.

## dependency

A2 had the 8 in it then changes into 2

Selected Cell: A2 = 8

|    | A  | B  | C   | D  | E | F | G | H | I | J |
|----|----|----|-----|----|---|---|---|---|---|---|
| 1  | 15 |    |     |    |   |   |   |   |   |   |
| 2  | 8  | 8  | 264 |    |   |   |   |   |   |   |
| 3  | 35 |    |     |    |   |   |   |   |   |   |
| 4  | 58 |    | 33  |    |   |   |   |   |   |   |
| 5  |    | 19 |     | 32 |   |   |   |   |   |   |
| 6  | 35 |    |     |    |   |   |   |   |   |   |
| 7  |    |    |     |    |   |   |   |   |   |   |
| 8  |    | 24 |     |    |   |   |   |   |   |   |
| 9  |    |    |     |    |   |   |   |   |   |   |
| 10 |    |    |     |    |   |   |   |   |   |   |

Enter value:2

Selected Cell: B2 = =MIN(A1..A4)

|    | A  | B  | C  | D  | E | F | G | H | I | J |
|----|----|----|----|----|---|---|---|---|---|---|
| 1  | 15 |    |    |    |   |   |   |   |   |   |
| 2  | 2  | 2  | 58 |    |   |   |   |   |   |   |
| 3  | 35 |    |    |    |   |   |   |   |   |   |
| 4  | 52 |    | 29 |    |   |   |   |   |   |   |
| 5  |    | 19 |    | 26 |   |   |   |   |   |   |
| 6  | 35 |    |    |    |   |   |   |   |   |   |
| 7  |    |    |    |    |   |   |   |   |   |   |
| 8  |    | 24 |    |    |   |   |   |   |   |   |
| 9  |    |    |    |    |   |   |   |   |   |   |
| 10 |    |    |    |    |   |   |   |   |   |   |

Selected Cell: A2 = 2

|    | A  | B  | C  | D  | E | F | G | H | I | J |
|----|----|----|----|----|---|---|---|---|---|---|
| 1  | 15 |    |    |    |   |   |   |   |   |   |
| 2  | 2  | 2  | 58 |    |   |   |   |   |   |   |
| 3  | 35 |    |    |    |   |   |   |   |   |   |
| 4  | 52 |    | 29 |    |   |   |   |   |   |   |
| 5  |    | 19 |    | 26 |   |   |   |   |   |   |
| 6  | 35 |    |    |    |   |   |   |   |   |   |
| 7  |    |    |    |    |   |   |   |   |   |   |
| 8  |    | 24 |    |    |   |   |   |   |   |   |
| 9  |    |    |    |    |   |   |   |   |   |   |
| 10 |    |    |    |    |   |   |   |   |   |   |